Introduction

Structural design patterns are concerned with the **composition of classes and objects**. They focus on how to assemble classes and objects into larger structures while keeping these structures flexible and efficient. Composite Pattern is one of the most important structural design patterns. Let's understand in depth.

Composite Pattern

The **Composite Pattern** is a structural design pattern that allows you to **compose objects into tree structures** to represent part-whole hierarchies. It lets clients treat individual objects and compositions of objects uniformly.

Problem It Solves

The Composite Pattern solves the problem of treating **individual objects** and **groups of objects** in the same way. The main problem arises when:

- You want to work with a hierarchy of objects.
- You want the client code to be **agnostic to whether it's dealing with a single object or a collection** of them.

Understanding the Problem

Consider you are building the checkout service of an e-commerce application and you take the following approach as shown in the code below.

Code (Without Composite Pattern)

```
#include <bits/stdc++.h>

using namespace std;
```

```
// Represents a single product
class Product {
private:
    string name;
    double price;

public:
    Product(string name, double price) : name(name), price(price) {}

    double getPrice() const {
        return price;
    }

    void display(const string& indent) const {
        cout << indent << "Product: " << name << " – ₹" << price << endl;
    }
};

// Represents a bundle of products
class ProductBundle {
```

```
private:
    string bundleName;
    vector<Product> products;

public:
    ProductBundle(string name) : bundleName(name) {}

    void addProduct(const Product& product) {
        products.push_back(product);
    }

    double getPrice() const {
        double total = 0;
        for (const auto& product : products) {
            total += product.getPrice();
        }
        return total;
    }

    void display(const string& indent) const {
        cout << indent << "Bundle: " << bundleName << endl;
```

```
        for (const auto& product : products) {
            product.display(indent + "  ");
        }
    }
};

// Main logic
int main() {
    // Individual Items
    Product book("Book", 500);
    Product headphones("Headphones", 1500);
    Product charger("Charger", 800);
    Product pen("Pen", 20);
    Product notebook("Notebook", 60);

    // Bundle: iPhone Combo
    ProductBundle iphoneCombo("iPhone Combo Pack");
    iphoneCombo.addProduct(headphones);
    iphoneCombo.addProduct(charger);

    // Bundle: School Kit
```

```
ProductBundle schoolKit("School Kit");
schoolKit.addProduct(pen);
schoolKit.addProduct(notebook);

// Add to cart logic
vector<void*> cart;
vector<string> cartType;

cart.push_back(&book);
cartType.push_back("Product");

cart.push_back(&iphoneCombo);
cartType.push_back("Bundle");

cart.push_back(&schoolKit);
cartType.push_back("Bundle");

// Display Cart
double total = 0;
cout << "Cart Details:\n" << endl;
```

```
    for (size_t i = 0; i < cart.size(); ++i) {
        if (cartType[i] == "Product") {
            Product* p = static_cast<Product*>(cart[i]);
            p->display("  ");
            total += p->getPrice();
        } else {
            ProductBundle* b = static_cast<ProductBundle*>(cart[i]);
            b->display("  ");
            total += b->getPrice();
        }
    }

    cout << "\nTotal Price: ₹" << total << endl;
    return 0;
}

import java.util.*;

// Represents a single product
class Product {
    private String name;
```

```
    private double price;

    public Product(String name, double price) {
        this.name = name;
        this.price = price;
    }

    public double getPrice() {
        return price;
    }

    public void display(String indent) {
        System.out.println(indent + "Product: " + name + " - ₹" + price);
    }
}

// Represents a bundle of products
class ProductBundle {
    private String bundleName;
    private List<Product> products = new ArrayList<>();
```

```
public ProductBundle(String bundleName) {
    this.bundleName = bundleName;
}

public void addProduct(Product product) {
    products.add(product);
}

public double getPrice() {
    double total = 0;
    for (Product product : products) {
        total += product.getPrice();
    }
    return total;
}

public void display(String indent) {
    System.out.println(indent + "Bundle: " + bundleName);
    for (Product product : products) {
        product.display(indent + "  ");
    }
```

```
    }
}

// Main logic
class Main {
    public static void main(String[] args) {
        // Individual Items
        Product book = new Product("Book", 500);
        Product headphones = new Product("Headphones", 1500);
        Product charger = new Product("Charger", 800);
        Product pen = new Product("Pen", 20);
        Product notebook = new Product("Notebook", 60);

        // Bundle: Iphone Combo
        ProductBundle iphoneCombo = new ProductBundle("iPhone Combo Pack");
        iphoneCombo.addProduct(headphones);
        iphoneCombo.addProduct(charger);

        // Bundle: School Kit
        ProductBundle schoolKit = new ProductBundle("School Kit");
        schoolKit.addProduct(pen);
```

```
schoolKit.addProduct(notebook);


// Add to cart logic
List<Object> cart = new ArrayList<>();
cart.add(book);
cart.add(iphoneCombo);
cart.add(schoolKit);


// Display Cart
double total = 0;
System.out.println("Cart Details:\n");


for (Object item : cart) {
   if (item instanceof Product) {
      ((Product) item).display("  ");
      total += ((Product) item).getPrice();
   } else if (item instanceof ProductBundle) {
      ((ProductBundle) item).display("  ");
      total += ((ProductBundle) item).getPrice();
   }
}
```

```
        System.out.println("\nTotal Price: ₹" + total);
    }
}

# Represents a single product
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

    def getPrice(self):
        return self.price

    def display(self, indent):
        print(f"{indent}Product: {self.name} – ₹{self.price}")

# Represents a bundle of products
class ProductBundle:
    def __init__(self, bundleName):
```

```
        self.bundleName = bundleName
        self.products = []

    def addProduct(self, product):
        self.products.append(product)

    def getPrice(self):
        return sum(product.getPrice() for product in self.products)

    def display(self, indent):
        print(f"{indent}Bundle: {self.bundleName}")
        for product in self.products:
            product.display(indent + "  ")


# Main logic
if __name__ == "__main__":
    # Individual Items
    book = Product("Book", 500)
    headphones = Product("Headphones", 1500)
    charger = Product("Charger", 800)
```

```
pen = Product("Pen", 20)

notebook = Product("Notebook", 60)


# Bundle: iPhone Combo

iphoneCombo = ProductBundle("iPhone Combo Pack")

iphoneCombo.addProduct(headphones)

iphoneCombo.addProduct(charger)


# Bundle: School Kit

schoolKit = ProductBundle("School Kit")

schoolKit.addProduct(pen)

schoolKit.addProduct(notebook)


# Add to cart logic

cart = [book, iphoneCombo, schoolKit]


# Display Cart

total = 0

print("Cart Details:\n")


for item in cart:
```

```
        if isinstance(item, Product):
            item.display("  ")
            total += item.getPrice()
        elif isinstance(item, ProductBundle):
            item.display("  ")
            total += item.getPrice()

    print(f"\nTotal Price: ₹{total}")
```

Working of Code

- `Product` class represents a simple item with `name` and `price`.
- `ProductBundle` class represents a group of products bundled together.
- Both classes have methods to display and return their prices.
- In `main()`, individual products and bundles are created and added to the cart.
- The cart is a `List<Object>` that holds both products and bundles.
- During checkout, the code checks each item's type using `instanceof`.
- Based on the type, it casts the object and calls its respective methods.
- Finally, it displays all items and calculates the total price.

Problem in above code

In the above example, the code lacks the structure to treat individual and group items uniformly, i.e., In the current implementation, **individual products** (`Product`) and **product bundles** (`ProductBundle`) are completely separate types with no shared interface or superclass. This means we cannot write code that treats both uniformly and the logic always has to check which type we're working with.

Other than these, there are some other problems as well:

- `instanceof` is used repeatedly, breaking polymorphism.
- Cart uses `List<Object>`, which is unsafe and violates abstraction.
- ProductBundle cannot contain another ProductBundle (no recursive structure).
- Display and price logic are duplicated instead of unified.

Refactored Code Using Composite Pattern

Let's refactor the code using the Composite Pattern. The idea is to create a common interface `CartItem` for both `Product` and `ProductBundle`, allowing us to treat them uniformly.

```
#include <bits/stdc++.h>

using namespace std;


// Interface for items that can be added to the cart

class CartItem {

public:

   virtual double getPrice() const = 0;
```

```
    virtual void display(const string& indent) const = 0;
    virtual ~CartItem() {}
};

// Product class implementing CartItem
class Product : public CartItem {
private:
    string name;
    double price;

public:
    Product(const string& name, double price) : name(name), price(price)
{}

    double getPrice() const override {
        return price;
    }

    void display(const string& indent) const override {
        cout << indent << "Product: " << name << " – ₹" << price << endl;
    }
};
```

```
// ProductBundle class implementing CartItem
class ProductBundle : public CartItem {
private:
    string bundleName;
    vector<CartItem*> items;

public:
    ProductBundle(const string& name) : bundleName(name) {}

    void addItem(CartItem* item) {
        items.push_back(item);
    }

    double getPrice() const override {
        double total = 0;
        for (const auto& item : items) {
            total += item->getPrice();
        }
        return total;
    }
```

```
    void display(const string& indent) const override {
        cout << indent << "Bundle: " << bundleName << endl;
        for (const auto& item : items) {
            item->display(indent + "  ");
        }
    }

    ~ProductBundle() {
        for (auto& item : items) {
            delete item;
        }
    }
};

// Main logic
int main() {
    // Individual Products
    CartItem* book = new Product("Atomic Habits", 499);
    CartItem* phone = new Product("iPhone 15", 79999);
    CartItem* earbuds = new Product("AirPods", 15999);
```

```
    CartItem* charger = new Product("20W Charger", 1999);

    // Combo Deal
    ProductBundle* iphoneCombo = new ProductBundle("iPhone Essentials Combo");
    iphoneCombo->addItem(phone);
    iphoneCombo->addItem(earbuds);
    iphoneCombo->addItem(charger);

    // Back to School Kit
    ProductBundle* schoolKit = new ProductBundle("Back to School Kit");
    schoolKit->addItem(new Product("Notebook Pack", 249));
    schoolKit->addItem(new Product("Pen Set", 99));
    schoolKit->addItem(new Product("Highlighter", 149));

    // Add everything to cart
    vector<CartItem*> cart;
    cart.push_back(book);
    cart.push_back(iphoneCombo);
    cart.push_back(schoolKit);

    // Display cart
```

```
    cout << "Your Amazon Cart:" << endl;
    double total = 0;
    for (const auto& item : cart) {
        item->display("  ");
        total += item->getPrice();
    }

    cout << "\nTotal: ₹" << total << endl;

    // Cleanup
    for (auto& item : cart) {
        delete item;
    }

    return 0;
}

import java.util.*;

// Interface for items that can be added to the cart
interface CartItem {
```

```
    double getPrice();
    void display(String indent);
}

// Product class implementing CartItem
class Product implements CartItem {
    private String name;
    private double price;

    public Product(String name, double price) {
        this.name = name;
        this.price = price;
    }

    @Override
    public double getPrice() {
        return price;
    }

    @Override
    public void display(String indent) {
```

```
        System.out.println(indent + "Product: " + name + " - ₹" + price);
    }
}

// ProductBundle class implementing CartItem
class ProductBundle implements CartItem {
    private String bundleName;
    private List<CartItem> items = new ArrayList<>();

    public ProductBundle(String bundleName) {
        this.bundleName = bundleName;
    }

    public void addItem(CartItem item) {
        items.add(item);
    }

    @Override
    public double getPrice() {
        double total = 0;
        for (CartItem item : items) {
```

```
            total += item.getPrice();
        }
        return total;
    }

    @Override
    public void display(String indent) {
        System.out.println(indent + "Bundle: " + bundleName);
        for (CartItem item : items) {
            item.display(indent + "  ");
        }
    }
}

// Main class
class Main {
    public static void main(String[] args) {
        // Individual Products
        CartItem book = new Product("Atomic Habits", 499);
        CartItem phone = new Product("iPhone 15", 79999);
        CartItem earbuds = new Product("AirPods", 15999);
```

```
        CartItem charger = new Product("20W Charger", 1999);

        // Combo Deal
        ProductBundle iphoneCombo = new ProductBundle("iPhone Essentials Combo");
        iphoneCombo.addItem(phone);
        iphoneCombo.addItem(earbuds);
        iphoneCombo.addItem(charger);

        // Back to School Kit
        ProductBundle schoolKit = new ProductBundle("Back to School Kit");
        schoolKit.addItem(new Product("Notebook Pack", 249));
        schoolKit.addItem(new Product("Pen Set", 99));
        schoolKit.addItem(new Product("Highlighter", 149));

        // Add everything to cart
        List<CartItem> cart = new ArrayList<>();
        cart.add(book);
        cart.add(iphoneCombo);
        cart.add(schoolKit);

        // Display cart
```

```
        System.out.println("Your Amazon Cart:");
        double total = 0;
        for (CartItem item : cart) {
            item.display("  ");
            total += item.getPrice();
        }

        System.out.println("\nTotal: ₹" + total);
    }
}
```

```
from abc import ABC, abstractmethod

# Interface for items that can be added to the cart
class CartItem(ABC):
    @abstractmethod
    def getPrice(self):
        pass

    @abstractmethod
    def display(self, indent):
```

```
        pass


# Product class implementing CartItem
class Product(CartItem):
    def __init__(self, name, price):
        self.name = name
        self.price = price

    def getPrice(self):
        return self.price

    def display(self, indent):
        print(f"{indent}Product: {self.name} – ₹{self.price}")

# ProductBundle class implementing CartItem
class ProductBundle(CartItem):
    def __init__(self, bundleName):
        self.bundleName = bundleName
        self.items = []

    def addItem(self, item):
```

```
        self.items.append(item)

    def getPrice(self):
        return sum(item.getPrice() for item in self.items)

    def display(self, indent):
        print(f"{indent}Bundle: {self.bundleName}")
        for item in self.items:
            item.display(indent + "  ")

# Main logic
if __name__ == "__main__":
    # Individual Products
    book = Product("Atomic Habits", 499)
    phone = Product("iPhone 15", 79999)
    earbuds = Product("AirPods", 15999)
    charger = Product("20W Charger", 1999)

    # Combo Deal
    iphoneCombo = ProductBundle("iPhone Essentials Combo")
    iphoneCombo.addItem(phone)
```

```
iphoneCombo.addItem(earbuds)

iphoneCombo.addItem(charger)


# Back to School Kit

schoolKit = ProductBundle("Back to School Kit")

schoolKit.addItem(Product("Notebook Pack", 249))

schoolKit.addItem(Product("Pen Set", 99))

schoolKit.addItem(Product("Highlighter", 149))


# Add everything to cart

cart = [book, iphoneCombo, schoolKit]


# Display cart

print("Your Amazon Cart:")

total = 0

for item in cart:

    item.display("  ")

    total += item.getPrice()


print(f"\nTotal: ₹{total}")
```

Working of Refactored Code

- `CartItem` interface defines the common methods for both products and bundles.
- `Product` and `ProductBundle` classes implement the `CartItem` interface.
- The cart now holds a list of `CartItem`, allowing us to treat both products and bundles uniformly.
- The display and price calculation logic is simplified, as we no longer need to check types.

Understanding Leaf and Composite in the Composite Pattern

In the Composite Design Pattern, we categorize components into two main roles:

- **Leaf (Individual Object):** A Leaf is a simple, atomic object in the structure. It does not contain any child components. In our example:
  - `Product` is a Leaf.
  - It represents individual purchasable items like books, phones, pens, etc.
  - Implements `CartItem` and provides its own `getPrice()` and `display()` logic.
- **Composite (Container of Components):** A Composite is a complex object that can hold multiple CartItem objects, including both Leaf and other Composite objects. In our example:
  - `ProductBundle` is a Composite.
  - It can contain `Products` (leaves) and even other `ProductBundles` (nested composites).
  - Implements `CartItem` and delegates actions (`getPrice()` and `display())` to its children.

How it Solves the Issues

- **Uniform Treatment via Shared Interface (CartItem):** Now, both `Product` and `ProductBundle` implement `CartItem`, so the cart can contain any of them without special handling.
  This eliminates the need for type checking (`instanceof`).
- **Enables Polymorphism:** All operations like `getPrice()` and `display()` are defined in the `CartItem` interface, so they can be called uniformly on both products and bundles.
  This simplifies logic and improves code extensibility.
- **Recursive Composition Made Easy:** Bundles can now include other bundles or products seamlessly. This supports deeply nested combos or kits which is a common real-world scenario.
- **No Code Duplication:** The cart-handling logic like computing total and displaying items is written once and works for any `CartItem`.
  This promotes cleaner, DRY (Don't Repeat Yourself) code.

When to Use Composite Pattern

The Composite Pattern is particularly useful when:

- **You have a hierarchical structure:** Use the composite pattern when your objects form a tree-like structure (e.g., folders inside folders, or products inside bundles).
- **You want to treat individual and groups in the same way:** When operations on single items and collections of items should be uniform (e.g., calculating total price, displaying structure).
- **You want to avoid client-side logic to differentiate leaf and composite:** Let polymorphism handle the differences between

simple and composite objects, keeping client code clean and maintainable.

Advantages and Disadvantages

Pros:

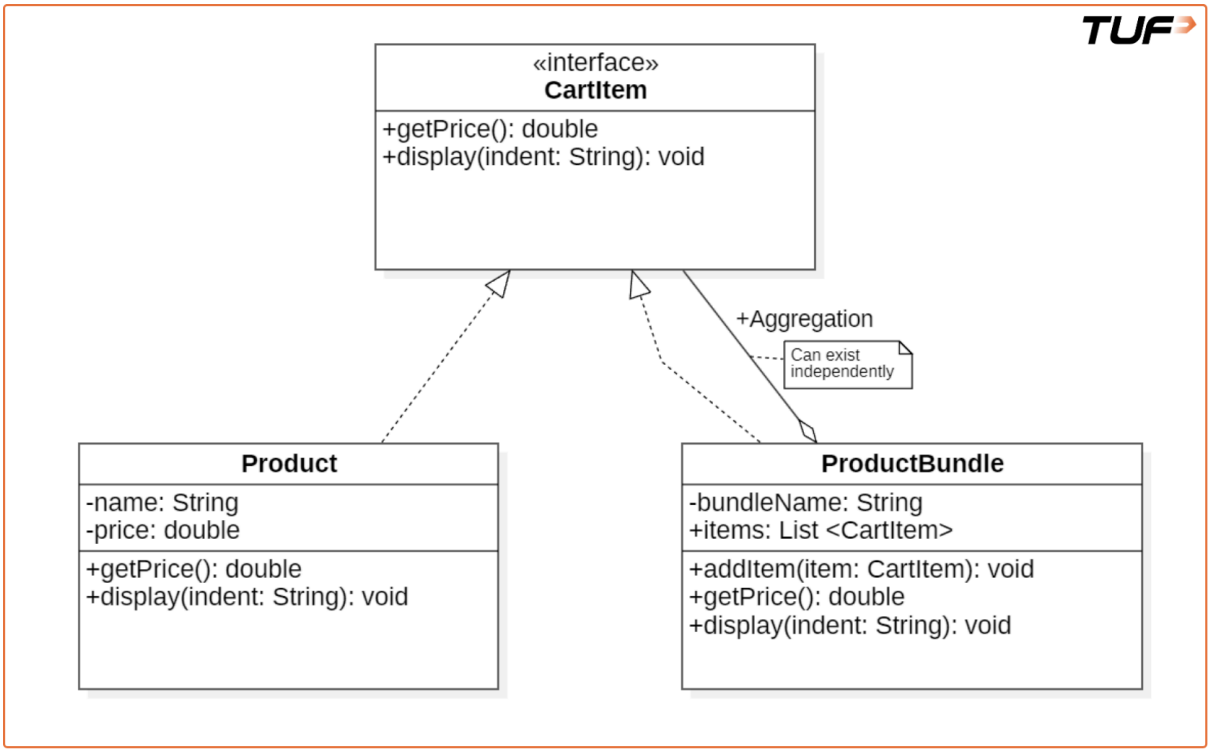
- **Uniformity:** Treats individual and composite objects in the same way.
- **Extensible:** Easy to add new item types or structures.
- **Cleaner client code:** Reduces complexity for the user of the structure.
- **Supports OCP (Open/Closed Principle):** Add new components without modifying existing code.

Cons:

- **Violates SRP on scale:** Components manage both hierarchy and business logic.
- **Overkill for flat and simple structures:** Adds unnecessary complexity.
- **Can hide important distinctions:** In regulated or sensitive systems, uniform treatment might blur critical differences between types.

Class Diagram

The class diagram below illustrates the structure of the Composite Pattern.


«interface»
CartItem
+getPrice(): double
+display(indent: String): void
+Aggregation
Can exist independently
Product
-name: String
-price: double
+getPrice(): double
+display(indent: String): void
ProductBundle
-bundleName: String
+items: List <CartItem>
+addItem(item: CartItem): void
+getPrice(): double
+display(indent: String): void